Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 17 tháng 8 2024, 9:38 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:59 PM
Thời gian thực hiện	6 Các ngày 6 giờ
Điểm	3,00/3,00
Điểm	**10,00** trên 10,00 (**100**%)

Câu hỏi 1

Đúng

Đạt điểm 1,00 trên 1,00

Implement function

```
int foldShift(long long key, int addressSize);
int rotation(long long key, int addressSize);
```

to hashing key using Fold shift or Rotation algorithm.

Review Fold shift:

The **folding method** for constructing hash functions begins by dividing the item into equal-size pieces (the last piece may not be of equal size). These pieces are then added together to give the resulting hash value.

For example:

Test	Result
cout << rotation(600101, 2);	26

Answer: (penalty regime: 0 %)

Reset answer

```
int foldShift(long long key, int addressSize)
{
    int sum=0;
    string s = to_string(key), substr="";
    int n=s.size();
    int divi=pow(10, addressSize);
    for (int i=0; i<n; i++) {
        substr+=s[i];
        if ((i+1) % addressSize == 0 || i==n-1) {
            sum+=stoi(substr);
            substr="";
        }
    }

    return sum%divi;

}

int rotation(long long key, int addressSize)
{
    long long remain = key%10, mul=1;
    long long newKey = key/10;
    while (key%mul < key) mul*=10;
    mul/=10;
    newKey+= remain*mul;
    return foldShift(newKey, addressSize);

}
```

	Test	Expected	Got	
✓	`cout << rotation(600101, 2);`	26	26	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **2**

Đúng

Đạt điểm 1,00 trên 1,00

Implement three following hashing function:

```
long int midSquare(long int seed);
long int moduloDivision(long int seed, long int mod);
long int digitExtraction(long int seed, int* extractDigits, int size);
```

Note that:

In midSquare function: we eliminate 2 last digits and get the 4 next digits.

In digitExtraction: extractDigits is a sorted array from smallest to largest index of digit in seed (index starts from 0). The array has size **size.**

For example:

Test	Result
int a[]={1,2,5}; cout << digitExtraction(122443,a,3);	223
cout <<midSquare(9452);	3403

Answer: (penalty regime: 0, 0, 0 %)

Reset answer

```
long int midSquare(long int seed)
{
    return ((seed*seed)/100)%10000;
}
long int moduloDivision(long int seed, long int mod)
{
    return seed%mod;
}
long int digitExtraction(long int seed,int* extractDigits,int size)
{
    string s=to_string(seed), result="";
    for (int i=0; i<size; i++) {
        result+=s[extractDigits[i]];
    }
    return stoi(result);
}

```

	Test	Expected	Got	
✓	int a[]={1,2,5}; cout << digitExtraction(122443,a,3);	223	223	✓
✓	cout <<midSquare(9452);	3403	3403	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **3**

Đúng

Đạt điểm 1,00 trên 1,00

There are `n` people, each person has a number between `1` and `100000` (`1 ≤ n ≤ 100000`). Given a number `target`. Two people can be matched as a **perfect pair** if the sum of numbers they have is equal to `target`. A person can be matched no more than 1 time.

Request: Implement function:

```
int pairMatching(vector<int>& nums, int target);
```

Where `nums` is the list of numbers of `n` people, `target` is the given number. This function returns the number of **perfect pairs** can be found from the list.

Example:

The list of numbers is `{1, 3, 5, 3, 7}` and `target = 6`. Therefore, the number of **perfect pairs** can be found from the list is `2` (pair `(1, 5)` and pair `(3, 3)`).

Note:

In this exercise, the libraries `iostream, string, cstring, climits, utility, vector, list, stack, queue, map, unordered_map, set, unordered_set, functional, algorithm` has been included and `namespace std` are used. You can write helper functions and classes. Importing other libraries is allowed, but not encouraged, and may result in unexpected errors.

For example:

Test	Result

`vector<int>items{1, 3, 5, 3, 7};`
`int target = 6;`
`cout << pairMatching(items, target);`
	2

`int target = 6;`
`vector<int>items{4,4,2,1,2};`
`cout << pairMatching(items, target);`
	2

Answer: (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
int pairMatching(vector<int>& nums, int target) {
    unordered_map<int, int> freqMap; // Map lưu trữ tần suất của các số
    int count = 0; // Đếm số cặp hoàn hảo

    // Duyệt qua tất cả các số trong nums
    for (int num : nums) {
        int complement = target - num; // Tính số bổ sung để tạo thành cặp với num

        // Kiểm tra nếu số bổ sung đã có trong map
        if (freqMap[complement] > 0) {
            count++; // Tìm thấy một cặp hoàn hảo
            freqMap[complement]--; // Giảm tần suất của số bổ sung
        } else {
            freqMap[num]++; // Thêm số hiện tại vào map
        }
    }

    return count;
}
```

	Test	Expected	Got	

✓	vector<int>items{1, 3, 5, 3, 7};
	int target = 6;
	cout << pairMatching(items, target);
		2	2	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.